

2025.12.5 15:00 over.

- 。 - 7, 27 or 17, 27, a 2-5501 to 5501,
- 。 説明を言葉で7に 27の2-7777, 図も7777. 読者には「イメージ」を共有する,

令和7年度 卒業論文

アニメーションを用いたSQL初学者向け 学習ツール SQL Quick Learning の 開発

指導教員

安藤 崇央 准教授

群馬大学 情報学部

情報学科 計算機科学プログラム

学籍番号 J2200152

宮下 柊司

令和7年1月

8

概 要

目次

第1章 はじめに	1
1.1 背景	1
1.1.1 社会的な需要	1
1.1.2 認知負荷と不可視性	2
1.1.3 既存の学習環境の限界	4
1.2 目的	4
1.2.1 提案システムの設計方針	5
1.2.2 評価の観点	5
1.3 関連研究	6
1.3.1 SQL 学習における誤解と困難性	6
1.3.2 可視化による学習支援システム	6
1.3.3 本研究の独自性	7
第2章 SQL Quick Learning について	8
2.1 システムの設計指針	8
2.1.1 クライアントサイド完結型のアーキテクチャ	8
2.1.2 ブロック組み立て方式によるクエリ構築	9

目次への書き込み

目 次	ii
2.1.3 状態遷移の可視化と制御	9
2.2 システム要件	10
2.2.1 機能要件	10
学習コンテンツ管理機能	10
クエリ構築機能 (ブロックエディタ)	11
SQL シミュレーション機能	12
実行過程可視化機能 (アニメーションプレイヤー)	12
正解判定機能	13
2.2.2 非機能要件	13
2.3 システム構成	14
2.4 実装詳細	14
2.5 ユーザインタフェース	14
第3章 実験と結果	15
3.1 実験方法	15
3.2 実験結果	15
第4章 まとめ	16
4.1 考察	16
4.2 今後の課題	16
謝 辞	17
参 考 文 献	18
付 録 A 理解度テスト	19

付録 B アンケート

20

第1章

はじめに

1.1 背景

1.1.1 社会的な需要

SQL(Structured Query Language) の習熟は、かつての専門的スキルという位置づけを超え、現在では多くの専門領域において必須の基礎技能となった。すなわち、SQL の習得は単なる言語学習ではなく、データが溢れる現代社会において構造化された思考と問題解決の手法を身につけるプロセスである。

現代の企業は、通販サイトから金融機関に至るまで、その運営基盤を構造化データに依存している。そして、SQL は、このデータから価値を引き出すための基本的な道具として、日常的な業務報告から高度なデータ分析、さらには機械学習の基盤構築まで、幅広い活動を支えている。具体的には、技術者にとって SQL は基礎スキルであり、一方で、データ分析者や営業管理者にとっては、自律的に業務を進め、より深い洞察を得るための道具となる。このように、組織内で誰もがデータに直接アクセスできる環境は、技術部門への依存を減らし、各担当者が必要な情報を迅速に入手できるため、結果とし

て業務効率を大きく向上させる。

このような SQL への需要は、単なる技術業界の流行ではなく、企業がデータに基づいて意思決定を行う必要性から生じる構造的な変化である。より詳しく述べると、この需要は二つの側面から生じている。一つは、大規模データや人工知能といった経営層主導の戦略的取り組みであり、これらは高度な分析のためのデータ準備に SQL を必要としている。もう一つは、業務効率化や自律性の向上といった現場からの要求である。したがって、この上下からの要求により、SQL は組織全体にわたって安定的かつ価値の高いスキルとなっている。言い換えれば、SQL を学ぶことは、技術職に就くためだけでなく、データを扱うほぼすべての業務において効果的に働くための要件となりつつある。

1.1.2 認知負荷と不可視性

「SQL は「宣言的」な
「プロセス」ではないか？」
「プロセス」は「宣言的」な
「プロセス」ではないか？」

高い需要に対し、SQL 学習における最大の障壁はその「宣言的」な性質にある。学習者は「何が欲しいか (What)」を記述するが、「それをどのように取得するか (How)」の具体的な実行プロセスはデータベース管理システムの最適化エンジンに委ねられる。この実行プロセス、例えば WHERE 句によるフィルタリング、JOIN によるテーブル結合、GROUP BY による集計がどのような順序で、どのような中間テーブルを生成しながら進むのかは、学習者にとって「不可視」である。

この不可視性こそが、認知負荷の源泉である。学習者は、この見えないプロセスと、それによって刻々と変化する中間データセットの変遷を、自らの「ワーキングメモリ内で視覚化する」という困難な精神的作業を強いられる。この精神的負荷が学習者の認知リソースを圧迫し、学習成果を著しく妨げる原因となっている。

Lister らによる大規模な研究 [1] は、約 1000 人の学生データを 8 年間にわたり分析し、初学者が直面する困難度を定量的に示した。その難易度は以下の通りである。

- (1) 1 テーブルからの単純なクエリ (最も容易)
- (2) GROUP BY (HAVING 句なし)
- (3) 自然結合 (Natural Join)
- (4) 単純なサブクエリ
- (5) GROUP BY (HAVING 句あり)
- (6) 相関サブクエリ
- (7) 自己結合 (Self Join) (最も困難)

この難易度階層は、単に構文の複雑さによるものではない。むしろ、学習者がワーキングメモリ内で同時に処理しなければならない「中間データセットの数とその複雑さ」の増加、すなわち認知負荷の増加率を反映している。SELECT ... FROM ... WHERE ... のような単純なクエリは一つの処理イメージで理解できるが、JOIN や GROUP BY が導入されると、学習者は複数の不可視な中間テーブルの生成と操作を脳内だけで追跡しなければならない。この点で認知負荷が爆発的に増加し、難易度の序列が形成されるのである。

また、初学者の誤りは、文法的な「構文エラー (Syntax Error)」よりも、構文的に正しいが意図した結果を返さない「セマンティックエラー (Semantic Mistake)」に集中していることが報告されている [2]。特に一般的なのは、問題解決に必要な句 (JOIN、GROUP BY 等) を記述しない「省略 (omission)」である。これは単なる知識不足ではなく、「問題の分解 (Problem Decomposition)」の失敗を示唆している。SQL 教育においては、構文だけでなく「どのような問題パターンが特定の句を必要とするのか」という問題認識スキルを明示的に指導する必要がある。

1.1.3 既存の学習環境の限界

上述した課題に対し、既存の学習環境は十分な解決策を提供できていないのが現状である。

第一に、教科書や静的な Web ページといった伝統的な教材は、図解を用いて概念を説明することはできるが、データの動的な変化を表現することには限界があり、直観的な理解を促すのが難しい。

第二に、Progate や SQLZOO といったインタラクティブな学習サービスは、実際にコードを実行できる点で優れているが、そのフィードバックは主に「最終結果 (Result Set)」の表示に留まっている。学習者にとって最も重要であるはずの「入力データがどのように加工されて最終結果に至ったか」というプロセスはブラックボックス化されたままであるため、SQL の動作原理を正しく理解することが困難である。

第三に、既存の環境では、初学者がスペルミスやカンマの忘れといった些細な構文エラーに多くの時間を費やしてしまう傾向がある。これにより、本来集中すべき論理構造やデータ操作のロジックの理解に認知リソースを割くことができず、学習効率の低下を招いている。

したがって、SQL 学習においては、中間プロセスの可視化によって認知負荷を低減させると同時に、構文エラーのノイズを排除し、論理構造の理解に特化できる新たな学習支援システムが求められている。

1.2 目的

本研究の目的は、SQL 初学者が直面する「記述順序と実行順序の乖離」および「中間プロセスの不可視性」という課題を解決する学習支援システムを開発し、その有効性を実験により検証することである。

。好みにもよるが、横書きでは、数（数を数えるための場合）は、算術数字を使うとよい。

例) 3種類. 第1章. 1つ, 2つ, ...

第1章 はじめに

ここは好みか ...

1.2.1 提案システムの設計方針

初学者の学習障壁を取り除くため、本システムでは以下の3点のアプローチを採用する。

? 「字面通り、前回の4章に解説している」というわけでは
よい といえるか?

第一に、論理実行順序の可視化である。SQLはその宣言的性質ゆえに、記述されたクエリ (SELECT ...) と実際の内部処理順序が異なる。本システムでは、FROM句によるデータの読み込み、WHERE句によるフィルタリング、GROUP BY句による集約といった一連の論理的なデータ操作プロセスを、ステップごとにアニメーションとして可視化する。これにより、従来不可視であった中間データセットの変遷を明示し、学習者が直観的に処理の流れを把握できる環境を提供する。

第二に、ブロックエディタの導入である。初学者は、スペルミスや記号の抜けといった些細な構文エラーの修正に多くの時間を費やしがちである。本システムでは、ドラッグ&ドロップでクエリを構築できるブロックエディタを採用し、構文エラーの発生を未然に防ぐ。これにより、学習者は「どのようなロジックでデータを操作するか」という本質的な理解に集中できる。

第三に、正しい動作イメージの構築支援である。上記の機能を通じて、学習者の頭の中に「SQLはデータに対してこのように作用する」という正確な理解を形成させ、誤った理解の固定化を防ぐ。

正しい構文エラーを防止?

1.2.2 評価の観点

表記を統一する。

開発したシステムの有効性を検証するため、以下の2つの観点から評価を行う。

第一に、学習効率および理解度の定量的評価である。アニメーション機能を有する本システムと、アニメーション機能を持たない環境との比較実験を行い、学習後のテストスコアや課題解決時間に有意な差が生じるかを検証する。

第二に、認知負荷の低減効果の定性的評価である。学習後のアンケート調査を通じ、可視化機能が学習者の精神的な負担感をどの程度軽減したかを確認する。

1.3 関連研究

1.3.1 SQL 学習における誤解と困難性

SQL の学習困難性に関しては、数多くの実証研究が行われている。Miedema ら [3] は、初学者の思考発話 (Think-aloud) プロトコルを用いた調査に基づき、SQL に関する誤解 (Misconceptions) を体系化した。Miedema は、初学者が抱く誤解の多くが、数学や手続きプログラミング言語といった「既存知識の誤った転移」や、データベースの動作原理に対する「不完全な動作イメージ」に起因すると指摘している。特に、宣言的な記述の背後にある手続き的な処理が見えないことが、正しい動作イメージの構築を妨げているとされる。本研究は、この動作イメージの構築を支援するための具体的なシステム開発を目的とするものである。

手続き型プログラミング言語

1.3.2 可視化による学習支援システム

プログラミング教育において、システムの内部動作を可視化することは有効な手段とされている。SQL の分野においても、いくつかの可視化ツールが提案されている。

Miedema らは、SQL クエリの構造をグラフとして可視化するツール『SQLVis』を開発した [3]。SQLVis は、テーブル間の結合関係やフィルタリング条件を視覚的に明示することで、複雑なクエリの解読を支援する。しかし、この手法はあくまで「記述された静的なコードの構造」を可視化するものであり、データが順次加工されている「動的な実行プロセス」を提示するものではない。

一方、実行プロセスの可視化という点では、Kearns らによる『esql』の研究 [4] が挙げられる。Kearns らは、SELECT 文の評価プロセスを「結合」「行の選択」「列の射影」といったステップに分解し、各段階における中間テーブル (Intermediate Tables) を順次表示するシステムを提案した。彼らの実験では、このステップごとの提示が、特に集約 (Aggregation) やグループ化 (Grouping) といった概念の理解に有効であることが示唆されている。しかしながら、『esql』はテキストベースの入力インタフェースを採用しており、学習者はロジックを確認する以前に、スペルミスや記号の誤りといった構文エラーの修正に時間を費やす必要がある。また、スタンドアロンのアプリケーションであるため、現代の学習者が手軽にアクセスできる Web ベースの環境ではない。

1.3.3 本研究の独自性

本研究で開発する『SQL Quick Learning』は、Kearns らが提唱した「論理実行順序のステップごとの可視化」というコンセプトを継承しつつ、現代的な Web 技術を用いたリッチなアニメーション表現へと発展させるものである。さらに、従来の研究では十分に解決されていなかった「構文エラーによる認知負荷」の問題に対処するため、ブロックエディタ形式の入力インタフェースを統合する。すなわち、「実行プロセスの可視化」によって動作イメージの構築を支援し、同時に「ブロックエディタ」によって構文の壁を取り除くという、二つのアプローチを統合した点に本研究の独自性と新規性が存在する。

1章の最後は「本論文の構成」のセクションが
あつたが普通かな。

第2章

SQL Quick Learning について

2.1 システムの設計指針

2.1.1 クライアントサイド完結型のアーキテクチャ

本システムは、モダンな Web アプリケーションフレームワークである Next.js を採用しつつ、従来の学習支援システムとは異なり、サーバーサイドのデータベース管理システムを持たない構成とした。SQL クエリの構文解析、論理実行順序に基づくシミュレーション、および結果の描画に至るすべての処理を、ブラウザ上で動作する JavaScript によって完結させている。このアーキテクチャには以下の利点がある。

- サーバー負荷の軽減
- 学習者の試行錯誤を支える即応性
- セキュリティリスクの低減

2.1.2 ブロック組み立て方式によるクエリ構築

SQL 学習における初期の障壁は、スペルミスや記号の欠落といった「構文エラー」である。初学者の多くは、本質的なデータ操作ロジックを理解する以前に、これらの形式的なエラー修正に時間を費やし、学習意欲を喪失する傾向がある。

本システムではこの課題を解決するため、テキスト入力を廃し、GUI ベースのブロック操作によるクエリ構築手法を採用した。具体的には、dnd-kit 等のライブラリを活用し、以下のインタラクションを実現している。

- 直観的な操作性
- 構文エラーの排除
- 制約の提示

「ブロック組み立て方式」を直観的に説明しやすい図かいい。

この設計により、学習者は構文の正確さを気にする認知負荷から解放され、「どのようなロジックでデータを操作するか」というセマンティックな理解に集中することが可能となる。

2.1.3 状態遷移の可視化と制御

既存の学習ツールの多くはクエリの「最終結果」のみを表示するが、これでは「なぜその結果になったのか」という内部プロセスの理解にはつながりにくい。本システムの革新的な設計思想は、SQL の論理的実行順 (FROM → JOIN → WHERE → GROUP BY ...) に沿って、データが段階的に加工されていく過程 (状態遷移) を可視化することにある。

本システムでは、シミュレーションロジックにより各処理ステップにおけるデータのスナップショットを「フレーム」として記録する。各フレームには、その時点でのデー

このプロセスを可視化しやすい図かいい。

タ行だけでなく、条件評価の結果やグループ化の状況といった「トランジション情報」も保持される。

アニメーション再生機能は、これらのフレーム配列を順次描画することで、データの流れを動画的に表現する。さらに、学習者が自身の理解度に合わせてペースを調整できるよう、以下の制御機能を提供する。

- 再生制御: 自動再生、一時停止、およびコマ送り/コマ戻し機能により、任意の処理ステップを繰り返し確認できる。
- 視覚的ハイライト: 実行中のブロックと、それによって影響を受けるデータ行を連動してハイライト表示することで、クエリ記述とデータ操作の因果関係を明示する。

この可視化と制御機能により、学習者はブラックボックス化されがちなデータベース管理システムの内部挙動を具体的なイメージとして捉え、SQL に対する深い理解を獲得することが期待できる。

2.2 システム要件

2.2.1 機能要件

本システムは、SQL の論理的な実行順序を初学者が直観的に理解できるように、以下の機能を提供する。

学習コンテンツ管理機能

subsubsection ?
副次にはなし ?

学習者が段階的にスキルを習得できるよう、カリキュラムを体系的に管理・提供する機能である。

・ itemize 環境ではなく description 環境を使う。

・ 「:」を日本語(全角)文字を使う。

- ・ 問題定義と管理: 各問題は、タイトル、説明文、使用可能なブロックパレット、操作対象のテーブルデータ、および模範解答を含む構造化データとして一元管理される。
item [問題 ... :] 各問題はー
- ・ トラック別カリキュラム: 問題は「基礎文法編」「集計関数編」「JOIN 編」といった単元に分類され、学習の進度に応じた適切な課題提示を行う。
- ・ レクチャーページ: 各問題の演習前に、当該単元の概念や構文を解説するレクチャーページを提供する。ここでは、SQL キーワードのハイライト表示等により、視覚的な理解を補助する。
- ・ 学習モード切替: アニメーションによる可視化を行う「通常モード」と、結果のみを表示する「静的モード」を切り替え可能とし、学習効果の比較検証に対応する。

クエリ構築機能 (ブロックエディタ)

フォントに違和感がある。
これ、どう書くの？

初心者が構文エラーに悩まされることなく、クエリの論理構造に集中できるよう、GUIベースのブロックエディタを提供する。

- ・ ドラッグ&ドロップ操作: パレット上のブロック (キーワード、列名、演算子等) をドラッグし、組み立てエリアに配置することでクエリを構築する。
- ・ ブロックのカテゴリ分類: ブロックは役割に応じて「キーワード」「列」「テーブル」「演算子」「関数」等に分類され、視覚的に区別される。
- ・ 使用制限と重複防止: FROM 句や LIMIT 句など、クエリ内で一度しか利用できない要素については、重複配置を UI レベルで防止する。

description 環境を使う。

「学習者」として あえて しゃべらなきゃ？

- リアルタイム SQL プレビュー: ブロックの配置状況に応じて、対応する SQL 文をリアルタイムに生成・表示し、学習者が自身の操作と SQL 構文の対応関係を確認できるようにする。

• この SQL の意味を
「SQL 構文の対応関係」を説明してくれる?

SQL シミュレーション機能

• 2.1.3 節の「フル」を説明する?

ブロックエディタで構築されたクエリを解析し、SQL の標準的な論理実行順序に従ってデータ処理をシミュレートする機能である。データベース管理システムを介さず、ブラウザ上の JavaScript エンジンで処理を完結させる。

- 論理実行順序の再現: FROM (データ読み込み) → JOIN (結合) → WHERE (フィルタリング) → GROUP BY (グループ化) → HAVING (グループフィルタリング) → SELECT (射影・集計) → DISTINCT (重複排除) → ORDER BY (並べ替え) → LIMIT (件数制限) という順序で、厳密にデータ加工を行う。
- 中間データの生成: 各処理ステップにおいて、その時点での「データスナップショット」と、どのような操作が行われたかを示す「トランジション情報」を生成し、アニメーションフレームとして記録する。
- エラーハンドリング: 必須句の欠落や不正なテーブル参照などの論理的な誤りを検出し、具体的なフィードバックを返却する。

実行過程可視化機能 (アニメーションプレイヤー)

シミュレータが生成した中間データに基づき、テーブルの変化をステップごとにアニメーションとして表示する機能である。

- 多様な視覚効果: 処理内容に応じて、視覚的フィードバックを行う。

- 再生コントロール: 自動再生、一時停止、コマ送り、コマ戻しの操作を提供し、学習者が自身の理解度に合わせて確認できるようにする。
- ブロックハイライト連動: 現在実行中の処理ステップに対応するクエリブロックをハイライト表示し、ロジックとデータ操作の因果関係を明示する。

正解判定機能

学習者が構築したクエリの実行結果と、あらかじめ定義された模範解答を比較し、正誤を判定する機能である。

- 厳密な結果比較: 行数、列構成、各セルの値を比較し、完全に一致した場合のみ正解とする。
- 具体的なフィードバック: 不正解の場合、「行数が一致しません」「列の構成が異なります」といった具体的なヒントを提示し、学習者の修正を支援する。

2.2.2 非機能要件

本システムの実用性と品質を担保するため、以下の非機能要件を定義する。

- ユーザビリティ: モダンな UI フレームワーク (Tailwind CSS) を採用し、統一感のあるデザインと直観的操作フィードバック（ホバー効果やドラッグ中の視覚効果など）を提供する。
- ポータビリティ: 特殊なサーバー環境やプラグインを必要とせず、一般的なモダン Web ブラウザ (Chrome, Edge, Firefox, Safari 等) のみで動作すること。

- レスポンス性能: すべての処理をクライアントサイドで行うことで、ネットワーク遅延の影響を受けず、ユーザー操作に対してリアルタイムに応答すること。
- 保守性と拡張性: 静的型付け言語 (TypeScript) とコンポーネント指向フレームワーク (React/Next.js) の採用により、コードの品質維持と将来的な機能拡張を容易にすること。

スポンサー

2.3 システム構成

2.4 実装詳細

2.5 ユーザインタフェース

第 3 章

実験と結果

3.1 実験方法

3.2 実験結果

第4章

まとめ

4.1 考察

4.2 今後の課題

謝 辞

謝辞をここに書く

参考文献

- [1] Alireza Ahadi, Julia R Prior, Vahid Behbood, and Raymond Lister. A quantitative study of the relative difficulty for novice of writing seven different types of sql queries. In *ITiCSE '15: Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education*, pp. 201–206, June 2015.
- [2] Alireza Ahadi, Julia Prior, Vahid Behbood, and Raymond Lister. Students' semantic mistakes in writing seven different types of sql queries. In *ITiCSE '16: Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, pp. 272–277, July 2016.
- [3] Daphne Eveline Miedema. *On Learning SQL: Disentangling concepts in Data Systems Education*. Phd thesis 1 (research tu/e / graduation tu/e), Mathematics and Computer Science, January 2024. Proefschrift.
- [4] R Kearns, S Sheard, and A. D. Fekete. A teaching system for sql. In *ACSE '97: Proceedings of the 2nd Australasian Conference on Computer Science Education*, pp. 224–231, July 1997.

付 録 A

理解度テスト

付 録 B

アンケート